



TRI

PAR

INSERTION

NDEYE COUMBA FALL

SOMMAIRE

INTRODUCTION

I. PRINCIPE ET MECANISME

II. ANALYSE DES PREFORMANCES : LIMITES ET
PUISSANCES

III. ENJEUX , APPLICATION

CONCLUSION

Bien que des outils et fonctions standards permettent de trier efficacement des données, il reste essentiel de comprendre les mécanismes internes des algorithmes de tri. Le tri par insertion, malgré une complexité en $O(n^2)$, occupe une place fondamentale dans l'apprentissage de l'algorithmique. Sa simplicité permet d'introduire des concepts clés tels que les invariants de boucle et l'analyse de complexité, faisant de cet algorithme une porte d'entrée privilégiée vers la pensée algorithmique, tout en conservant des applications pratiques pour de petits ensembles de données ou des données presque triées

INTRODUCTION



PRINCIPE DU TRI PAR INSERTION

```
#include <stdio.h>

void tri_insertion(int T[], int n) {
    int i, j, cle;

    // Boucle principale : on considère que T[0] est déjà trié
    for (i = 1; i < n; i++) {
        cle = T[i]; // Élément à insérer
        j = i - 1;

        // Décalage des éléments plus grands vers la droite
        while (j >= 0 && T[j] > cle) {
            T[j + 1] = T[j];
            j--;
        }

        // Insertion de la clé à sa position correcte
        T[j + 1] = cle;
    }
}
```

Le tri par insertion repose sur une idée simple et intuitive, comparable au tri d'une main de cartes. On considère que les éléments sont traités un par un, de gauche à droite. Au départ, la première partie de la liste est considérée comme déjà triée (au minimum, le premier élément).

À chaque étape, on sélectionne un nouvel élément appelé clé. Cette clé est ensuite comparée aux éléments de la partie déjà triée. Tant que ces éléments sont plus grands que la clé, ils sont décalés d'une position vers la droite afin de libérer l'emplacement correct. Une fois la bonne position trouvée, la clé est insérée.

ANALYSE DES PERFORMANCES

Le tri par insertion présente des performances étroitement liées à sa complexité et à l'organisation initiale des données. Bien que sa complexité moyenne et dans le pire cas soit en $O(n^2)$, ce qui le rend peu adapté aux grands ensembles de données, il se révèle très efficace lorsque les données sont déjà ou presque triées, avec un temps d'exécution proche de $O(n)$. Son exécution en place, sans mémoire supplémentaire, et sa bonne localité mémoire améliorent ses performances sur de petits tableaux. Ainsi, malgré ses limites théoriques, le tri par insertion reste un algorithme pertinent dans des contextes spécifiques et illustre parfaitement l'importance d'adapter le choix d'un algorithme au type de données traité.

```
int pire_cas[] = {6, 5, 4, 3, 2, 1};
```

```
int meilleur_cas[] = {1, 2, 3, 4, 5, 6};
```

```
int cas_moyen[] = {3, 1, 4, 2, 6, 5};
```



```
void tri_insertion(int T[], int n) {  
    // Seules 3 variables locales peu importe la taille de T  
    int i;      // 4 octets  
    int j;      // 4 octets  
    int cle;    // 4 octets  
    // TOTAL : 12 octets CONSTANTS  
  
    for (i = 1; i < n; i++) {  
        cle = T[i];  
        j = i - 1;  
        while (j >= 0 && T[j] > cle) {  
            T[j + 1] = T[j]; // Décalage DANS le tableau original  
            j--;  
        }  
        T[j + 1] = cle; // Insertion DANS le tableau original  
    }  
}
```

Tri "en place" (mémoire constante $O(1)$).

Un algorithme "en place" (in-place) n'utilise qu'une quantité constante de mémoire supplémentaire, indépendante de la taille des données en entrée.

Tri stable

Un tri est stable s'il préserve l'ordre relatif des éléments ayant la même clé de tri.

```
typedef struct {
    int note;      // Clé primaire de tri
    char nom[20];  // Information secondaire
    int id;        // Identifiant unique
} Etudiant;

Etudiant classe[] = {
    {15, "Alice", 1},
    {12, "Bob", 2},
    {15, "Charlie", 3}, // Même note qu'Alice
    {10, "David", 4}
};
```

Adaptatif

```
typedef struct {  
    int note;      // Clé primaire de tri  
    char nom[20];  // Information secondaire  
    int id;        // Identifiant unique  
} Etudiant;  
  
Etudiant classe[] = {  
    {15, "Alice", 1},  
    {12, "Bob", 2},  
    {15, "Charlie", 3}, // Même note qu'Alice  
    {10, "David", 4}  
};
```

Un algorithme est adaptatif s'il profite de l'ordre existant dans les données. Plus les données sont déjà partiellement triées, plus il est rapide.

Pour le tri par insertion :

- Meilleur cas (déjà trié) : $O(n)$ – linéaire
- Pire cas (tri inverse) : $O(n^2)$ – quadratique
- Cas quasi trié : proche de $O(n)$

ENJEUX ET APPLICATIONS

ENJEUX

Performance : $O(n^2)$ dans le pire cas (tableau inversé), mais $O(n)$ si déjà trié, ce qui pose un enjeu pour les données réelles souvent quasi-ordonnées.

Stabilité : Préserve l'ordre relatif des éléments égaux, un atout critique pour les tris multi-clés (ex. : tri par nom puis prénom) contrairement au tri par sélection.

Espace mémoire : Tri en place ($O(1)$), idéal pour les environnements contraints comme les microcontrôleurs.

Simplicité pédagogique : Parfait pour enseigner les invariants de boucle et la correction par récurrence

APPLICATIONS

- Listes liées : Efficace pour insérer dans des structures dynamiques comme les listes chaînées, où les décalages sont naturels.
- TimSort hybride : Intégré dans les bibliothèques de tri de Python, Java et .NET pour gérer les runs pré-triés dans les données réelles.
- Systèmes embarqués : Utilisé dans les jeux de cartes numériques, agendas ou petites bases de données sur appareils limités (montres, capteurs IoT).
- Optimisations avancées : Combiné à la recherche dichotomie pour $O(n \log n)$ comparaisons, ou dans les shells sort pour de meilleurs résultats pratiques.

CONCLUSION

Le tri par insertion démontre qu'un algorithme simple reste très utile en pratique, particulièrement pour les petites données ou tableaux presque triés malgré sa complexité $O(n^2)$. La théorie pure ne suffit pas toujours : un tel algorithme peut surpasser des solutions $O(n \log n)$ dans des contextes réels spécifiques. Au-delà, il enseigne des concepts fondamentaux comme les invariants de boucle, la stabilité et l'adaptativité, applicables à tous les tris avancés.



THANK YOU !!!